

Mobile Software Development Group Assignment Proposal

Group 1: Ike Marsters, Daniel Saunders, Sifa Zhang, Amelie Hemmerich

Contents

Mobile Application Name: GrabYourGear	1
Architectural front-end and back-end	1
Application Screen Design	2
Description of Initial Implementation	2
Complications	2
Acquiring Knowledge and Skills	3
Difficult Android/Kotlin/Java Concepts	3
Achievability	4
Risks	5
Initial Task Distribution	5
User Cases for each User Role	6

Mobile Application Name: GrabYourGear

For this project, we have decided to develop an Equipment Rental App called *GrabYourGear*.

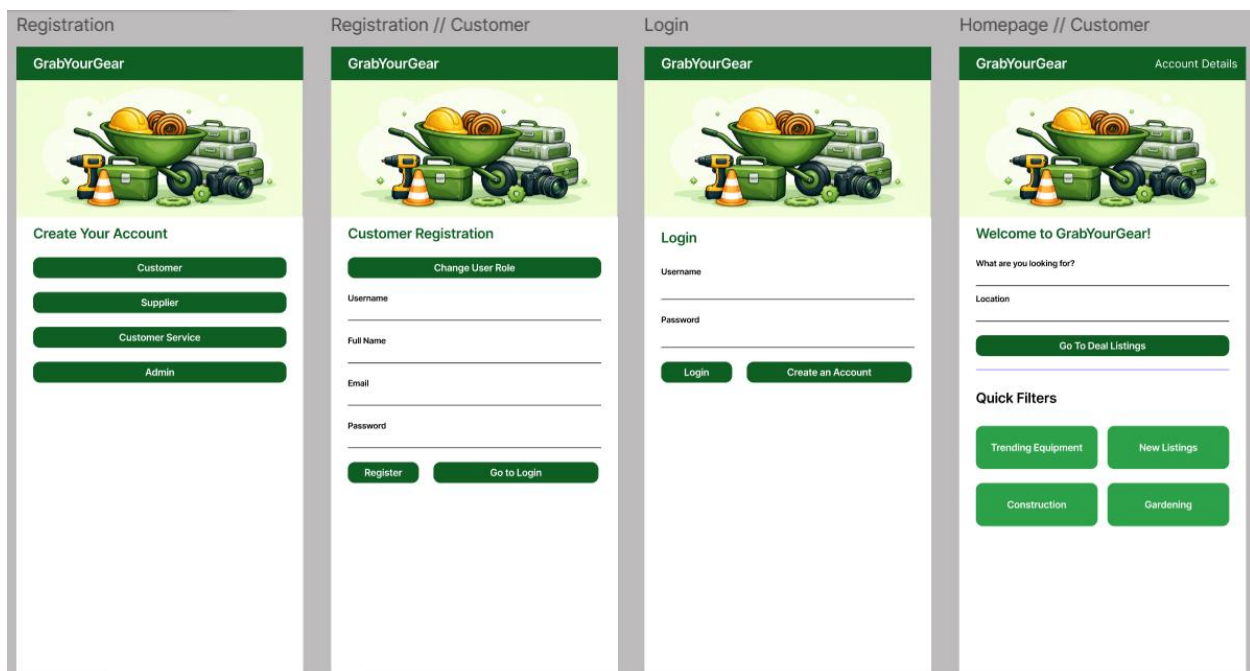
Architectural front-end and back-end

The frontend will be implemented in Android Studio using Java and will handle the user interfaces and navigation between the different screens aiming for a user-friendly experience. The backend will be based on Firebase, including authentication and the cloud database to manage user data and the application content.

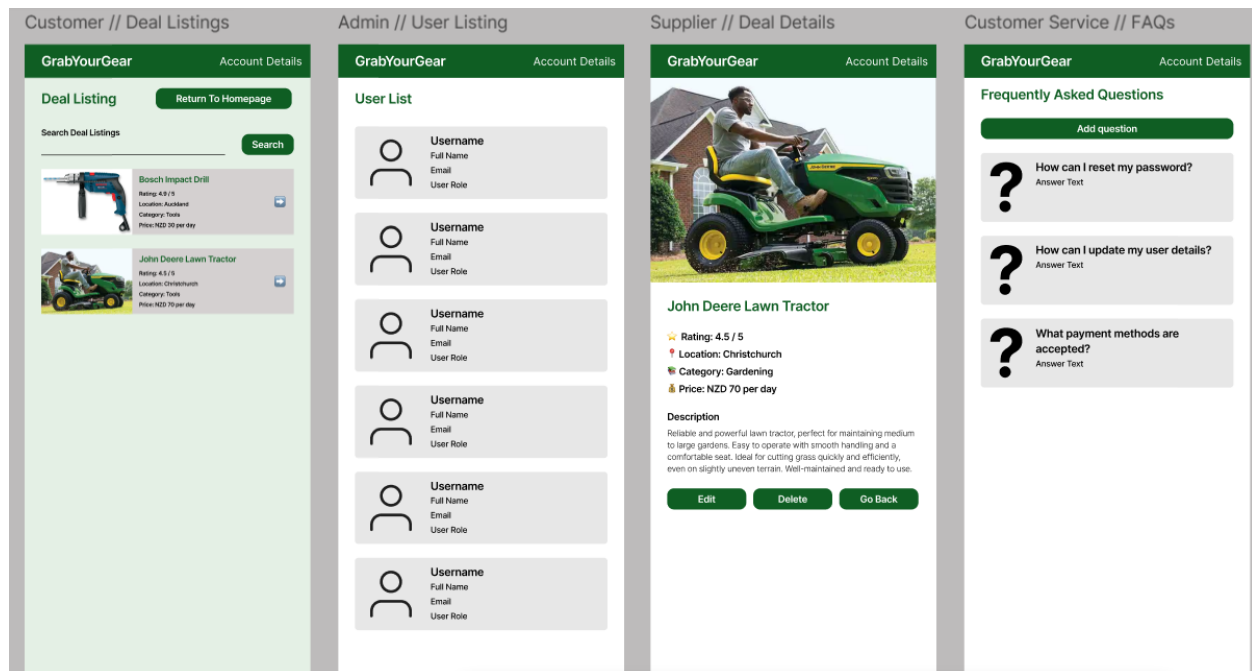
Application Screen Design

In the following screenshots, you can see a first design for some of the screens. These screen designs were created in FIGMA.

The first screenshot shows screen designs for the registration and login process as well as a first design for the homepage for customers.



In the second screenshot, you can see screen designs for different use cases per user role. The first screen on the left shows the deal listings page for the customers. The next one shows the user listings page that can be accessed by the admin user. The third screen is an example for the deal details page for the suppliers, where they can edit or delete their product listing. Finally, the screen on the right shows a page for the frequently asked questions that are updated by the customer service role.



Description of Initial Implementation

The initial implementation will focus on setting up the structure of the application, including the project structure in Android studio and the integration of key technologies such as Firebase for our cloud storage. First, we need to work on the user interface designs and the basic navigation between those screens. Furthermore, we need to prototype basic features for each user role, such as registration and login and the specific user dashboards. Additionally, we will need to create a database structure in Firebase and connect it to our project.

All in all, this serves as a starting point for continued implementation and development of our final product.

Complications

1. Concurrency and Data Consistency Issues

The Realtime Database uses a single large JSON tree, which makes concurrency control more challenging. When multiple customers attempt to book the same equipment at the same time, or when a supplier updates availability while a booking is being created, conflicting writes may occur. These require careful design to prevent race conditions and double bookings. Ensuring atomic updates in a real-time environment is one of the key technical challenges.

2. NoSQL Data Structure Design Complexity

The Realtime Database stores all data in a hierarchical JSON structure. This requires the team to carefully design structure nodes for users, equipment, and bookings, avoid deeply nested data, prevent circular references, and minimise data duplication while still supporting efficient queries. Because Realtime Database does not support queries as flexibly, the team must make more design decisions to balance performance, maintainability, and data consistency.

3. Testing Challenges

Testing the system is complex due to the involvement of multiple user types, real-time database updates, and potential concurrency scenarios. Ensuring that booking logic, permissions, and data synchronization behave correctly across different devices and network conditions requires thorough and well-structured testing.

4. Team Collaboration and Version Control Difficulties

Developing an Android application as a team introduces coordination challenges. Merge conflicts in Git, overlapping changes to UI resources, and inconsistent updates to Firebase Security Rules can disrupt progress. Effective communication and disciplined version control practices are essential to avoid integration issues.

Acquiring Knowledge and Skills

For this project, we will need both knowledge and experience in Android Studio. Java will primarily be the programming language for this project, however some SQL will likely be needed for data handling. Furthermore, we need to gain experience with Firebase, for features such as authentication and interacting with our cloud database.

Difficult Android/Kotlin/Java Concepts

Since our application will be developed using Java and Android Studio, there are several important concepts that our team may need to learn or strengthen in order to complete the project successfully. The following are the two most significant areas:

1. Asynchronous Programming with Firebase

All Firebase operations—including authentication, reading/writing data, and uploading images—are asynchronous. In Java, this requires learning how to work with Firebase callbacks, success and failure listeners, handling asynchronous flows without blocking the UI, and managing multi-step operations (e.g., upload image → get download URL → save booking data). This is essential for implementing booking logic, updating equipment availability, and ensuring a smooth and responsive user experience.

2. Realtime Database Data Modeling and NoSQL Structure

The Firebase Realtime Database uses a hierarchical JSON tree, which requires a different approach compared to traditional SQL databases. We need to understand how to design an efficient JSON structure for users, equipment, bookings, and suppliers, avoid deeply nested data that is difficult to maintain, prevent unnecessary data duplication, query data efficiently despite RTDB's limited querying capabilities, manage relationships without JOINS, and ensure consistency when multiple user roles interact with the same data. This is one of the more challenging parts of the project because NoSQL databases require more design decisions and careful planning to avoid data inconsistency and performance issues..

Achievability

Given the project scope, team size, and available timeframe, the application is achievable as long as the team follows a structured development plan and divides responsibilities effectively.

1. Team Size Matches the Number of User Roles

With four developers and four user roles (Customer, Supplier, Admin, and Customer service), each team member can take ownership of one role's UI and functionality. This creates a natural and efficient division of work, reducing overlap and minimizing coordination overhead.

2. Firebase Greatly Reduces Backend Complexity

Since the project uses Firebase (Authentication, Realtime Database, Storage), the team does not need to build or maintain a custom backend server. This significantly shortens development time because Firebase handles user authentication, data storage, real-time updates, and file/image uploads. This allows the team to focus mainly on Android UI and business logic.

3. Core Features Are Manageable Within the Timeline

The main features—equipment listing, booking, user management, and basic CRUD operations—are straightforward to implement with Firebase. These are common patterns with plenty of documentation and examples, making development faster.

4. Parallel Development Is Possible

Because each user's role has its own screens and workflows, the team can work in parallel without blocking each other. This increases productivity and reduces bottlenecks.

5. Risks Are Manageable

The most complex parts of the system (e.g., concurrency, booking logic, and Realtime Database structure) are well understood and can be implemented incrementally. As long as the team agrees on the data structure early, the rest of the development becomes much smoother.

Overall, the application is achievable within the given timeframe. With four developers, a clear division of responsibilities, and Firebase handling the backend, the team is well-positioned to complete the project by June 2. The key to success will be early agreement on data structure, consistent communication, and a focus on core features before adding enhancements.

Risks

While working on this project, there are several risks we need to keep in mind. Since we only have a limited amount of time to complete this project, one risk might be that we may not be able to meet deadlines or that certain features we planned can't be implemented in time. Furthermore, since this is the first time this team has worked together in a group, it might lead to insufficient team coordination or communication. Additionally, there may also be technical difficulties, for example when trying to get the different parts – frontend and backend - of the application to work together smoothly.

All in all, these risks can be managed through careful planning and clear communication.

Initial Task Distribution

For this project, each team member will be responsible for developing the interface and backend for one of the four user roles.

Sifa will focus on the *Customer* user role, covering features such as account management, browsing and renting equipment.

Ike will implement the *Admin* role, including user management and overall platform management.

The *Supplier* user role will be covered by Amelie, responsible for product management and inventory related features.

Finally, the *Customer Service* role will be handled by Daniel, focusing on support features such as handling inquiries, complaints and more.

This distribution ensures efficient development and clear responsibilities.

User Cases for each User Role

Customer - Use Cases:

- **Browse and Search Equipment:** The customer can view all available equipment, search by keywords, and filter by category, price, or availability.
- **Make a Booking for Equipment:** The customer selects a piece of equipment and submits a booking request for specific dates.
- **View and Manage Bookings:** The customer can see all their current and past bookings, including status (pending, confirmed, cancelled), date and price. The customer can cancel a booking if allowed.

Admin - Use Cases:

- **Approve/Deny Supplier Applications:** Administrators will oversee approving or denying a supplier when they register to the platform; suppliers will undergo an approval process which concludes with a platform administrator approving or denying the request.
- **User Management:** Admins must be able to view a list of users with the ability to, for example, ban users when required.
- **Supplier Management:** Admins must be able to view a list of all suppliers registered with the app with the ability to perform administrative tasks such as deleting suppliers or altering supplier information.

Supplier - Use Cases:

- **Create Product Listings:** Suppliers can create a new product listing including details such as description, price and availability.
- **Update Product Information:** Suppliers can update or edit the product details in case there are any changes.
- **Manage Orders:** Suppliers can view and process incoming requests from users for their products.

Customer Service - Use Cases:

- **Manage Frequently Asked Questions:** Customer service can provide answers to frequently asked questions and support the user regarding products or account issues.

- Handle Inquiries: Customer service can respond to any email inquiries sent through an inquiry form.
- Handle User Feedback: Customer service can read through any user feedback forms and deliver them to the admins for improving the application.